



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统前沿论文阅读报告

GeminiFS: 为 GPU 设计的配套文件系统

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：崔立骁

2026 年 5 月 31 日

摘要

当前基于 GPU 的存储解决方案能够通过 NVMe 队列让 GPU 直接访问存储设备，这打破了原有基于 CPU 的存储方案的高 CPU-GPU 同步开销、I/O 流量放大以及较高的 CPU 处理延迟所带来的性能瓶颈。然而，新兴的存储方案缺少类似传统主机文件系统所提供的文件抽象或管理功能。本文提出了一种专为 GPU 设计的配套文件系统，并在大规模机器学习工作负载场景下，性能远超当前最先进的存储解决方案。

关键字：GeminiFS，元数据，页面缓存，GPU 编程，SNVMe

目录

一、 研究背景	1
二、 GeminiFS 介绍	1
(一) 设计原理	1
(二) 实施方案	2
三、 实验评估	3
四、 总结与展望	5

一、 研究背景

随着图神经网络（GNN）和大语言模型（LLM）等 GPU 加速的机器学习应用的快速发展，其处理的数据集规模和模型权重参数日益庞大，通常达到数十 TB 且持续增长。尽管 GPU 内存容量在过去十年间有所提升，但仍难以满足 ML 应用日益增长的需求。传统的基于内存的扩展方案，如 DRAM，虽然能缓解压力，但在大规模场景下成本高昂。相比之下，基于存储的 GPU 内存扩展方案更具成本效益，且随着高性能闪存技术的发展，其性能损耗较小。然而，如何高效地让 GPU 访问这些存储设备成为了新的挑战。

现有的 GPU 存储访问解决方案主要分为两类，包括以 CPU 为中心的方案和以 GPU 为中心的方案，其中前者为主要研究方向，主要成果包括 Dragon，GPUfs 和 GDS，通过直接依赖 CPU 显式或隐式地发起对文件的访问，其性能瓶颈显著，问题关键在于高并发场景下，CPU 相较于 GPU 并行程度低，致高昂的 CPU-GPU 同步开销和 I/O 流量放大。GPUfs 和 GDS 的软件栈开销占比极高（超过 90%），且随着 GPU 线程数增加，延迟急剧上升。

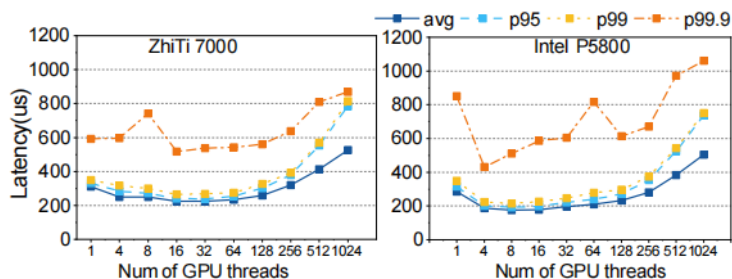


图 1: Average and tail latencies of 4KB read I/O for GPUfs under different numbers of threads. As the number of threads increases, the average and tail latencies significantly increase due to contention among CPU cores.

在以 GPU 为中心的存储访问解决方案中，最具代表性的为 BaM，其通过将控制平面完全实现在 GPU 上，通过在 GPU 内存中分配 NVMe 队列，使 GPU 线程能直接向 NVMe 设备发送 I/O 命令，绕过 CPU 的限制，从而实现高吞吐量和细粒度访问。这样的做法相比于先前的方案的确取得了较大的性能提升，但是仍然面临较大的性能瓶颈：

- 缺乏文件抽象与管理层：只提供块设备接口，不支持传统主机文件系统提供的文件抽象和访问控制功能。
- 数据共享困难：元数据隔离使得不同 GPU 进程之间以及 GPU 与 CPU 进程之间的数据共享变得困难。

基于上述现实背景，本文的研究者提出了一种既能绕过 CPU 直接访问存储设备（GPU-centric），又能提供文件系统抽象和管理功能的解决方案，以此解决上述两种方案的显著问题。

二、 GeminiFS 介绍

（一） 设计原理

GPU 加速应用的两个关键特性，是 GeminiFS 设计的底层原理：

- I/O 可预测性：GPU 存储 I/O 工作负载具有一定可预测性，存储访问信息可根据模型设置提前获取。

- 元数据特性：大多数磁盘数据在其生命周期内为只读，且写入操作多为追加写，元数据相对稳定。

因此，GeminiFS 的思想是“配套文件系统”，即不构建完整的 GPU 文件系统，而是与主机文件系统共存并配合工作。在主机端，由主机文件系统负责文件的创建、移动、删除等全部管理操作，并将必要的文件私有元数据（如文件大小、类型、访问模式、逻辑块到物理块的映射）直接嵌入到文件本身；而在 GPU 端，仅需在文件打开时检索并缓存这些嵌入的元数据，基于此为 GPU 程序提供文件系统抽象和读写接口。这样的设计避免了在 GPU 端维护复杂元数据的巨大开销，又通过元数据共享实现了 CPU 与 GPU 文件系统之间高效且安全的同步。

为了保证 GPU 直接、高效地访问存储，GeminiFS 做出了三个关键创新尝试：

- 嵌入式块映射: GeminiFS 通过在文件创建时，将逻辑块到 NVMe 物理块的映射关系作为元数据嵌入文件，使 GPU 在读取数据时能够直接根据文件偏移量计算出 NVMe 偏移量，从而绕过了复杂的地址转换逻辑，提升 I/O 效率。
- CPU/GPU 共享 NVMe 驱动: GeminiFS 对驱动程序进行了扩展，新增了 GPU 缓冲区管理模块。该模块记录 GPU 内存分配，并借助 GPU 驱动 API 将分配在 GPU 内存中的 I/O 队列内存页持锁定，并转换为 DMA 地址，使其对 NVMe 设备可见。
- GPU 专用页面缓存: 在主机端的 SNVMe 中设置了页缓存管理模块，首次打开文件时分配 GPU 内存并获取跨进程内存句柄，后续进程打开同文件时即可通过句柄共享同一页缓存。

（二） 实施方案

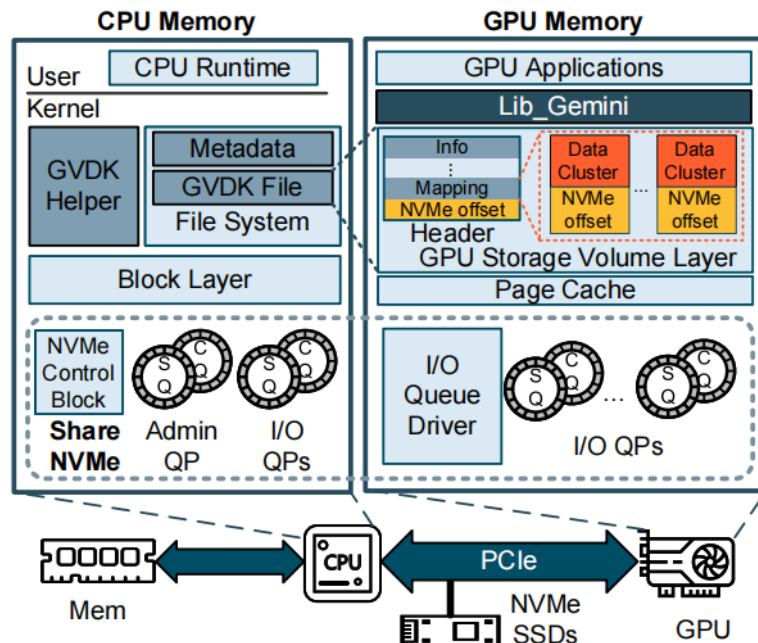


图 2: Overview of GeminiFS

图片展示了 GeminiFS 的整体架构：分为 CPU 内存和 GPU 内存两大域，通过 PCIe 总线与 NVMe SSDs 互联：CPU 内存的用户态包含负责系统初始化和资源管理的 CPU Runtime，内核态则由生成嵌入元数据的 GVDK 格式文件的 GVDK Helper 模块、管理磁盘空间和全局元数据

的主机文件系统、提供统一块设备访问接口的块层, 以及核心的共享 NVMe 驱动 (SNVMe) 组成, SNVMe 包含 NVMe 控制块、管理队列和 CPU 侧 I/O 队列; GPU 内存部分最上层是运行机器学习任务的 GPU 应用程序, 下方是抽象底层复杂性、提供类 POSIX 文件接口的 libGemini 库, 再往下是缓存文件元数据和两级块映射表、实现文件偏移到 NVMe 物理偏移快速转换的 GPU 存储卷层, 利用 GPU 高带宽加速重复数据访问的 GPU 专用页缓存, 以及最底层的 GPU 侧 I/O 队列驱动和多个 GPU 侧 I/O 队列, 允许 GPU 线程直接向 NVMe 设备提交 I/O 命令并通过轮询方式完成操作; 整个架构中 CPU 负责复杂的文件系统管理和全局资源控制, GPU 专注于高性能文件 I/O, 数据通过 DMA 直接在 NVMe SSD 和 GPU 内存之间传输, 完全绕过 CPU, 同时通过将元数据嵌入文件的方式实现了主机与 GPU 文件系统的高效同步。

三、实验评估

在完成对 GeminiFS 的设计后, 研究者开展了相应的性能评估实验, 以回答以下问题:

- 与最先进的解决方案相比, GeminiFS 在性能上有哪些优势?
- 如何在 GPU 架构上最大化 GeminiFS 的性能?
- GeminiFS 如何为实际应用带来益处?

所有实验在配备 64 核 Intel Xeon 5416S 处理器、512GB 内存、运行 Ubuntu 20.04 及 Linux 内核 5.15.0 的服务器上进行; 该服务器搭载 80GB HBM 的 GPU (峰值带宽 1,935 GB/s), 通过 PCIe Gen4 x16 互联 (带宽 64 GB/s), 并连接一块挂载 EXT4 文件系统的 Intel Optane 5800X NVMe 存储设备; NVMe 控制器支持最多 135 对 I/O 队列对 (QP), 其中主机分配 64 个 QP, GPU 分配 32 个 QP, 块大小均设为 4KB。

研究者首先与 SOTA 方案进行基础性能对比:

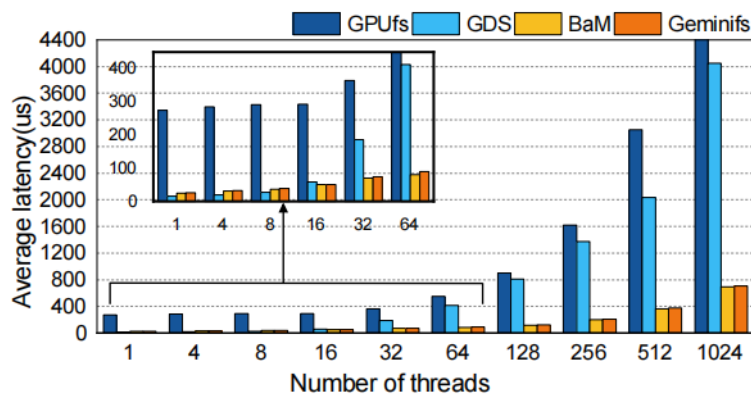


图 3: Comparison of 4K read I/O average latency of GeminiFS with GPUfs, GDS and BaM at various levels of parallelism.

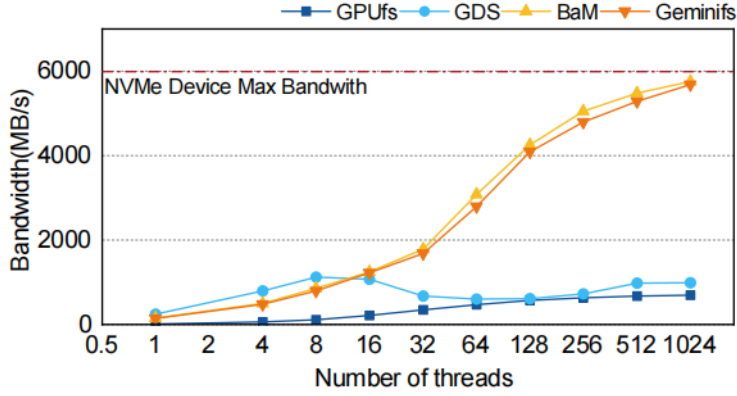


图 4: Comparison of 4K read I/O bandwidth of GeminiFS with GPUfs, GDS and BaM at various levels of parallelism.

从图中可以明显看出, GeminiFS 在高并发 I/O 场景下展现出显著优势, 其创新性的 GPU 端直接 I/O 处理架构打破了 CPU 瓶颈。实验数据表明, 随着线程数的增加, GeminiFS 的带宽性能呈指数级增长, 充分释放了 NVMe 存储设备的硬件潜力。相较于 GPUfs, 带宽提升 7.33 倍, 相较于 GDS, QPS 提升 5.2 倍, 且相较于直接对裸设备进行操作 BaM, 性能损耗基本可忽略不计。

在延迟 (latency) 方面, 在高并发 I/O 场景下, GeminiFS 显著降低了数据读写的等待时间, 相比传统方案实现了接近一个数量级的延迟优化。相较于 GPUfs, 延迟下降 90%, 相较于 GDS, 延迟下降 83%, 相较于 BaM, 仅上升 4.8%, 随着并发线程数的增加, 传统方案的延迟呈现指数级增长, 而 GeminiFS 的延迟曲线始终保持平缓。这种稳定性对于大规模并行计算至关重要, 意味着在处理海量数据时, 系统能够维持高效且可预测的响应速度, 从根本上提升了上层应用的吞吐量与执行效率。

随后, 在真实的 GPT2-124M 模型训练场景中, GeminiFS 凭借底层存储架构的创新设计, 突破了传统训练框架的性能上限。通过对 Checkpoint 机制与数据流转路径的深度优化, 该方案在不同卸载策略下均实现了显著的耗时缩减, 为大规模 LLM 训练的效率提升提供了强有力的技术支撑。

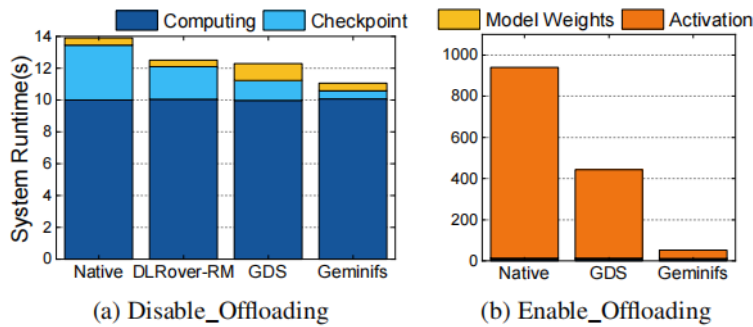


图 5: Comparison of GPT2-124M training performance between GeminiFS, GDS, DLRouter-RM and a native way (memcpy + read/write). (a) Maintain the activation on HBM; (b) Offload the activation.

在禁用激活卸载 (Disable Offloading) 场景下, 运行时间缩减 25%, 在无需显存卸载的基础训

练模式下,相比原生方案显著提升了系统吞吐量与迭代速度;在启用激活卸载 (Disable Offloading) 场景下,运行时间相较原生耗时下降 94.5%,较 GDS 方亦降低 91%,在大模型训练加速方面发挥了极好的性能。

四、 总结与展望

本文提出了面向 GPU 协同的 GeminiFS 配套文件系统,创新性实现了 GPU-centric 存储方案,打破传统限制让 GPU 通过文件接口直接访问主机文件系统管理的磁盘空间,重构了异构计算下的存储交互模式,通过依托 GPU 架构缩短存储控制与数据路径,在保障与主机文件系统共存兼容的基础上显著提升 I/O 性能,精准匹配机器学习等数据密集型应用对高吞吐、低延迟存储访问的核心需求。

研究者指出,在未来 GeminiFS 将全面支持多 GPU 环境,其技术路线图主要包括以下几个方面:首先,将通过逻辑分割文件的方式来实现并行读写;其次,采用 RAID 技术整合多个 NVMe 存储设备,以满足多 GPU 场景下的高带宽需求;针对不可预测的工作负载,GeminiFS 将引入文件预分配机制,即在系统运行前预先分配文件槽位,并在运行时根据实际使用情况批量分配文件以填充这些槽位;此外,计划将 GeminiFS 集成到 PyTorch 框架中,以便 vLLM 等前沿解决方案能够从中受益。

本文选自第 23 届 USENIX 文件与存储技术会议论文集,在论文阅读过程中,我学到了如何就现有的存储方案的性能瓶颈出发,通过研究其底层原理,发现 CPU 并行程度低导致的高并发场景性能下降,从而转而考虑相应的优化措施,对于现有技术的缺陷,提出相应的解决方案用于完善。针对 GPU 存储访问中“CPU 协调模式低效”与“GPU 直接访问缺乏文件抽象”的矛盾,可以通过“配套文件系统”的理念折中解决。

论文提到 GVDK 格式基于“大部分数据是追加或只读”的假设。如果遇到密集随机写或频繁的元数据修改,主机的 GVDK Helper 模块会不会成为新的瓶颈?此时元数据嵌入的更新开销如何?此外,论文主要基于 NVIDIA CUDA 生态,这种深度的驱动修改方案如何迁移到 AMD 或 Intel 等平台?是否存在硬件抽象层的阻碍?这些都是未来论文可能探索的方向。